

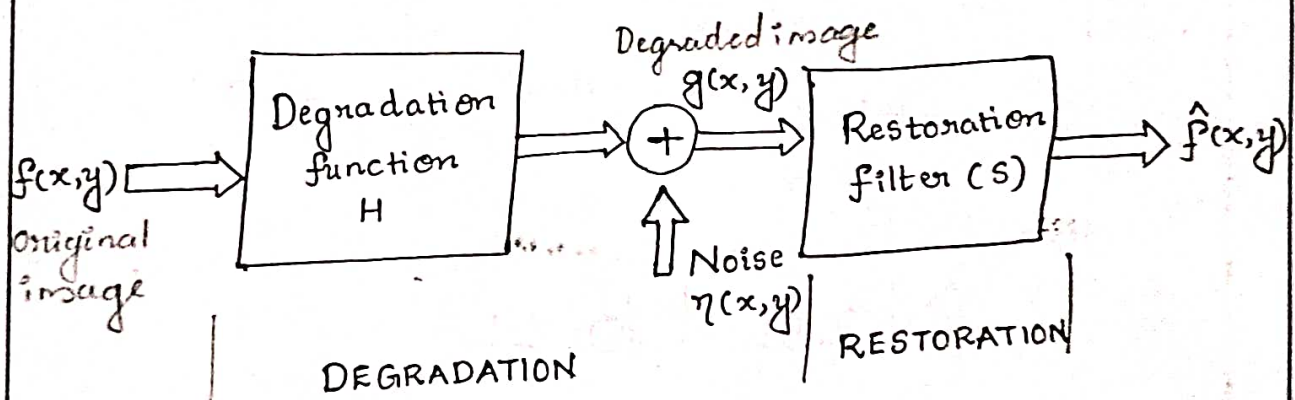
MODULE - 4IMAGE RESTORATION AND
IMAGE SEGMENTATIONIMAGE DEGRADATION MODEL

Figure 1: A model of the image degradation/restoration process.

In the figure: it operates on an input image $f(x,y)$ to produce a degraded image $g(x,y)$, some knowledge about the degradation function H , and some knowledge about the additive noise term $\eta(x,y)$, the objective of restoration

is to obtain an estimate $\hat{f}(x, y)$ of the original image. The degraded ^{kind of blurring} image is given in the spatial domain by

$$g(x, y) = h(x, y) \star f(x, y) + n(x, y) \quad (1)$$

' $h(x, y)$ ' is the spatial representation of the degradation function. ^{Additive noise}

' $\star$ ' indicates convolution.

Eqn (1) in the equivalent frequency domain representation:

$$G(u, v) = H(u, v) F(u, v) + N(u, v) \quad (2)$$

$$F(u, v) = G(u, v) / H(u, v) - N(u, v) / H(u, v)$$

NOISE MODELS

The following are among the most common Probability density function (PDF) found in image processing applications. Pdf defines the distribution of noise in an image.

1. Gaussian Noise

The PDF of a Gaussian random variable, x is given by

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\bar{x})^2}{2\sigma^2}}$$

- Affects both dark & white (light) areas of image.

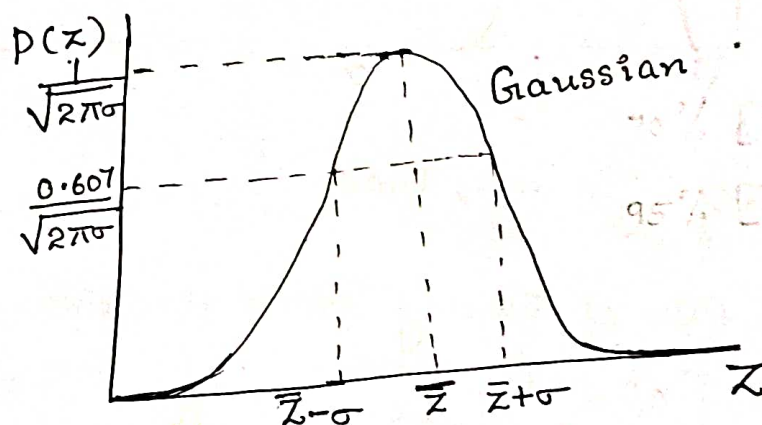
Where,

Z represents intensity (gray level)

$\bar{Z}$ is the mean (average) value of Z .

σ is its standard deviation.

The standard deviation squared σ^2 , is called the variance of Z .



- Most commonly used model.

2. Rayleigh Noise

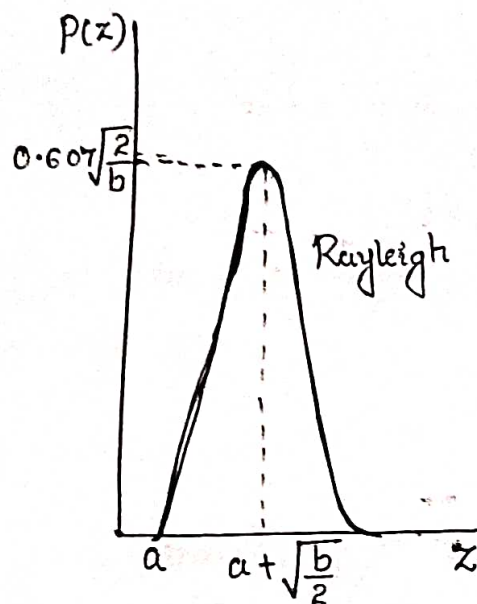
The PDF of Rayleigh noise is given by a, b ; non-negative integers

$$P(Z) = \begin{cases} \frac{2}{b}(z-a)e^{-(z-a)^2/b} & ; \text{ for } z \geq a \\ 0 & ; \text{ for } z < a \end{cases}$$

The mean and variance of this density are given by

$$\text{Mean, } \bar{Z} = a + \sqrt{\pi b/4}$$

$$\text{Variance, } \sigma^2 = \frac{b(4-\pi)}{4}$$



3. Erlang (gamma) Noise

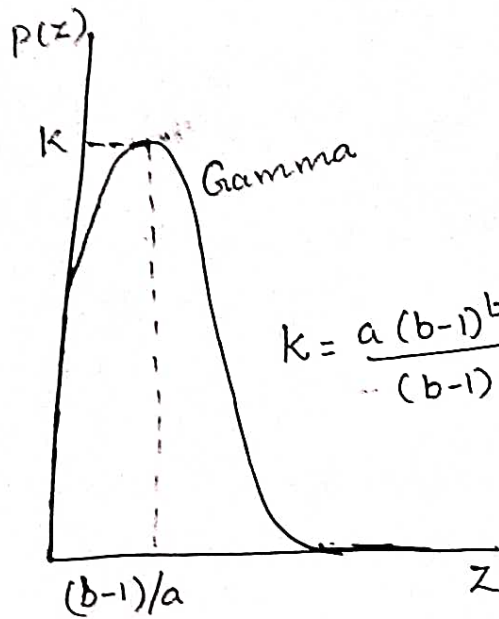
The PDF of Erlang noise is given by

$$P(z) = \begin{cases} \frac{a^b z^{b-1}}{(b-1)!} e^{-az} & ; \text{for } z \geq 0 \\ 0 & ; \text{for } z < 0 \end{cases}$$

Where the parameters are such that $a > 0$, b is positive integer and "!" indicates factorial. The mean and variance of this density are given by

$$\text{Mean } \bar{z} = \frac{b}{a}$$

$$\text{Variance } \sigma^2 = \frac{b}{a^2}$$



$$k = \frac{a(b-1)^{b-1}}{(b-1)!} e^{-(b-1)}$$

$\gamma(b)$ when then gamma noise.

4. Exponential Noise

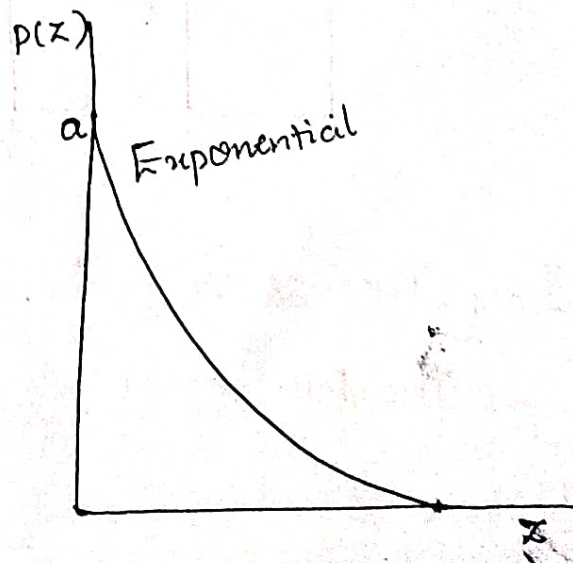
The PDF of exponential noise is given by

$$P(z) = \begin{cases} ae^{-az} & ; \text{for } z \geq 0 \\ 0 & ; \text{for } z < 0 \end{cases}$$

Where $a > 0$.

Mean, $\bar{z} = \frac{1}{a}$

Variance, $\sigma^2 = \frac{1}{a^2}$



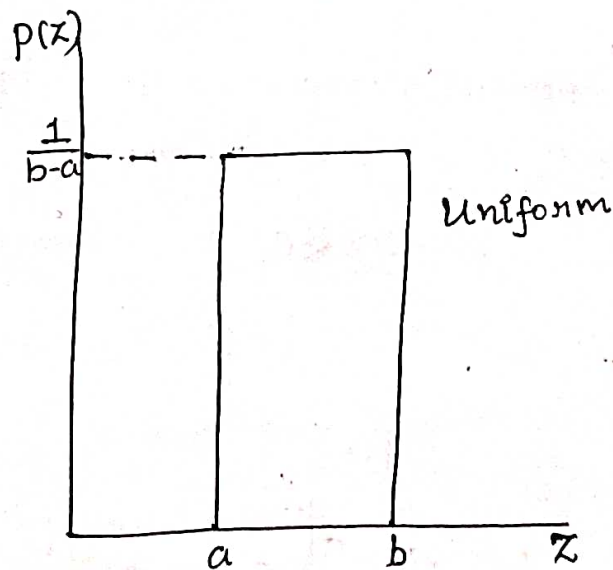
5. Uniform Noise

The PDF of uniform noise is given by

$$p(z) = \begin{cases} \frac{1}{b-a} & \text{if } a \leq z \leq b \\ 0 & \text{otherwise} \end{cases}$$

$$\text{Mean, } \bar{z} = \frac{a+b}{2}$$

$$\text{Variance, } \sigma^2 = \frac{(b-a)^2}{12}$$



6. Impulse (Salt-and-Pepper) Noise

The PDF of (bipolar) impulse noise is given by:

- Binary noise

$$P(z) = \begin{cases} P_a & ; \text{ for } z=a \\ P_b & ; \text{ for } z=b \\ 0 & ; \text{ otherwise} \end{cases}$$

If $b > a$, intensity b will appear as a light dot in the image.

If either P_a or P_b is zero, the impulse noise is called unipolar.

MEAN FILTERS

* [The noise reduction capabilities of the spatial filters are discussed in smoothing spatial filter (studied in S6 CG&IP)] Various mean filter use are:

1. Arithmetic Mean Filter
2. Geometric Mean Filter
3. Harmonic mean filter
4. Contraharmonic Filter.

1. Arithmetic Mean Filter:

This is the simplest of the mean filters.

Let $S_{x,y}$ represent the set of coordinates in a rectangular subimage window (neighbourhood) of size $m \times n$

Centered at point (x, y) . The arithmetic mean filter computes the average value of the corrupted image $g(x, y)$ in the area defined by S_{xy} . The value of the restored image $\hat{f}$ at point (x, y) is simply the arithmetic mean computed using the pixels in the region defined by S_{xy} .

In other words,

$$\hat{f}(x, y) = \frac{1}{mn} \sum_{(s, t) \in S_{xy}} g(s, t)$$

This operation can be implemented using a spatial filter of size $m \times n$ in which all coefficients have value $1/mn$. A mean filter smooths local variations in an image, and noise is reduced as a result of blurring.

2. Geometric Mean Filter.

An image restored using a geometric mean filter is given by the expression.

$$\hat{f}(x, y) = \left[\prod_{(s, t) \in S_{xy}} g(s, t) \right]^{\frac{1}{mn}}$$

Here each restored pixel is given by the product of the pixels in the subimage window, raised to the power $1/mn$.

3. Harmonic mean filter

The harmonic mean filtering operation is given by the expression

$$\hat{f}(x,y) = \frac{mn}{\sum_{(s,t) \in S_{xy}} \frac{1}{g(s,t)}}$$

The harmonic mean filter works well for salt and Gaussian noise, but fails for pepper noise.

4. Contraharmonic mean filter

The contraharmonic mean filter yields a restored image based on the expression

$$\hat{f}(x,y) = \frac{\sum_{(s,t) \in S_{xy}} g(s,t)^{Q+1}}{\sum_{(s,t) \in S_{xy}} g(s,t)^Q}$$

Where Q is called the order of the filter. This filter is well suited for reducing or virtually eliminating the effects of salt-and-pepper noise.

$$Q = \begin{cases} +ve & ; \text{eliminates pepper noise} \\ -ve & ; \text{salt} \end{cases}$$

ORDER - STATIC FILTERS

Order-static filters are spatial filters whose response is based on ordering (ranking) the values of the pixels contained in the image area encompassed by the filter. The ranking result determines the response of the filter.

1. Median Filter

The best-known order-static filter is the median filter, which, as its name implies, replaces the value of a pixel by the median of the intensity level in the neighborhood of that pixel.

$$\hat{f}(x, y) = \text{median}_{(s, t) \in S_{xy}} \{g(s, t)\}$$

2. Max and min filters

Using the 100th percentile results in the so-called max filter, given by

$$\hat{f}(x, y) = \max_{(s, t) \in S_{xy}} \{g(s, t)\}$$

The 0th percentile filter is the min filter,

$$\hat{f}(x, y) = \min_{(s, t) \in S_{xy}} \{g(s, t)\}.$$

The max filter is useful for finding the brightest points in an image. The min filter is useful for finding the darkest points in an image. Also, it reduces salt noise as a result of the min operation.

3. Midpoint Filter

The midpoint filter simply computes the midpoint between the maximum and minimum values in the area encompassed by the filter.

$$\hat{f}(x, y) = \frac{1}{2} \left[\max_{(s, t) \in S_{xy}} \{g(s, t)\} + \min_{(s, t) \in S_{xy}} \{g(s, t)\} \right]$$

This filter combines order statistics and averaging. It works best for randomly distributed noise, like Gaussian or uniform noise.

4. Alpha-trimmed mean filter

Suppose that we delete $d/2$ lowest and the $d/2$ highest intensity values of $g(s, t)$ in the neighborhood S_{xy} . Let $g_x(s, t)$ represent the remaining $m - d$ pixels. A filter formed by

A filter formed by averaging these remaining Pixels is called an alpha-trimmed mean filter.

$$\hat{f}(x, y) = \frac{1}{mn-d} \sum_{(s,t) \in S_{xy}} g_n(s, t)$$

Where the value of d can range from 0 to $mn-1$. When $d=0$, the alpha-trimmed filter reduces to the arithmetic mean filter.

If we choose $d=mn-1$, the filter becomes a median filter.

For other values of d , the alpha-trimmed filter is useful in situations involving multiple types of noise such as a combination of salt-and-pepper & Gaussian noise.

ADAPTIVE FILTERS

In adaptive filters, the behavior changes based on statistical characteristics of the image inside the filter region defined by the $m \times n$ rectangular window S_{xy} .

- Adaptive filters gives better performance than all other previous filters.

1. Adaptive, local noise reduction filter.

This filter is operating on local region S_{xy} . The response of the filter at any point (x, y) on which the region is centered is to be based on four quantities:

(a) $g(x, y)$, the value of noisy image at (x, y)

(b) σ_n^2 , the variance of the noise corrupting $f(x, y)$ to form $g(x, y)$

(c) m_L , the local mean of the pixel in S_{xy}

(d) σ_L^2 , the local variance of the pixels in S_{xy}

The behavior of the filter to be as follows:

1. If σ_n^2 is zero, the filter should return simply the value of $g(x, y)$.

$$g(x, y) = f(x, y); \text{ zero noise}$$

2. If the local variance is high relative to σ_n^2 the filter should return a value close to $g(x, y)$.

3. If the two variances are equal, then return the arithmetic mean value of the pixels in S_{xy} .

$$\hat{f}(x, y) = g(x, y) - \frac{\sigma_n^2}{\sigma_h^2} [g(x, y) - m]$$

2. Adaptive Median Filter.

The adaptive median - filtering algorithm works in two stages, denoted stage A and stage B, as follows:

Stage A:

$$A_1 = Z_{med} - Z_{min}$$

$$A_2 = Z_{med} - Z_{max}$$

If $A_1 > 0$ AND $A_2 < 0$, go to stage B

Else increase the window size.

If window size $\leq S_{max}$ repeat stage A

Else output Z_{med}

Stage B:

$$B_1 = Z_{xy} - Z_{min}$$

$$B_2 = Z_{xy} - Z_{max}$$

If $B_1 > 0$ AND $B_2 < 0$, Output Z_{xy}
Else output Z_{med} .

Where,

Z_{min} = minimum intensity value in S_{xy} .
 Z_{max} = maximum intensity value in S_{xy} .
 Z_{med} = median of intensity values in S_{xy} .
 Z_{xy} = intensity value at coordinates (x, y) .
 S_{max} = maximum allowed size of S_{xy} .

EDGE DETECTION

Edge detection is the process of finding meaningful transitions in an image. Edge detection is one of the central tasks of the lower levels of image processing. The points where sharp changes in the brightness occur typically form the border between different objects. These points can be detected by computing intensity differences in local image regions. Most of the edge detectors work on measuring the intensity gradient at a point in the image.



GRADIENT OPERATORS

A gradient is a two-dimensional vector that points to the direction in which the image intensity grows fastest. The gradient operator ∇ is given by

$$\nabla = \begin{bmatrix} \partial/\partial x \\ \partial/\partial y \end{bmatrix}$$

If the operator ∇ is applied to the function f then

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

The gradient magnitude gives the amount of the difference between pixels in the neighbourhood which gives the strength of the edge.

The gradient magnitude is given by

$$|\nabla f| = \left| \begin{bmatrix} G_x \\ G_y \end{bmatrix} \right| = \left| \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} \right|$$

$$= [G_x^2 + G_y^2]^{1/2}$$

The gradient orientation gives the direction of the greatest change, which presumably is the direction across the edge.

The gradient orientation is given by

$$\phi(\nabla f) = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

Where the angle is measured with respect to the x-axis.

An edge pixel is described using two important features:

- (i) Edge strength, which is equal to the magnitude of the gradient.
- (ii) Edge detection, which is equal to the angle of the gradient.

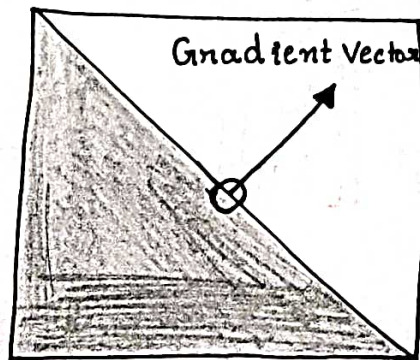


Fig: Gradient of edge pixel.

LAPLACE OPERATORS & ZERO CROSSINGS

Finding the ideal edge is equivalent to finding the point where the derivative is maximum or minimum.

The maximum or minimum value of a function can be computed by differentiating the given function and finding places where the derivative is zero.

- * Differentiating the first derivative gives the second derivative.
- * Finding the optimal edge is equivalent to finding places where the second derivative is zero.
- * The differential operators can be applied to images;
- * The zeros rarely fall exactly on a pixel, typically they fall between pixels.

★ The zeros can be isolated by finding the zero crossings.

Zero crossing is the place where one pixel is positive and neighbouring pixel is negative.

The problems with zero-crossing methods are the following:

- (i) Zero crossing methods produce two-pixel thick edges.
- (ii) Zero crossing methods are extremely sensitive to noise.

For images, there is a single measure, similar to the gradient magnitude, that measures the second derivative, which is obtained by taking the dot product of ∇ with itself.

$$\nabla \cdot \nabla = \begin{bmatrix} \partial/\partial x \\ \partial/\partial y \end{bmatrix} \cdot \begin{bmatrix} \partial/\partial x \\ \partial/\partial y \end{bmatrix}$$

$$\nabla \cdot \nabla = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$

The operator $\nabla \cdot \nabla = \nabla^2$ is called the Laplacian operator.

The Laplacian operator when applied to the function f , we get

$$\nabla^2 f = \left(\begin{bmatrix} \partial/\partial x \\ \partial/\partial y \end{bmatrix} \cdot \begin{bmatrix} \partial/\partial x \\ \partial/\partial y \end{bmatrix} \right) f$$

$$= \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

The Laplacian operation can be expressed in terms of difference equations as given below:

$$\frac{\partial f}{\partial x} = f(x+1, y) - f(x, y)$$

and

$$\frac{\partial^2 f}{\partial x^2} = f(x+1, y) - 2f(x, y) + f(x-1, y)$$



Also,

$$\frac{\partial^2 f}{\partial y^2} = f(x, y+1) - 2f(x, y) + f(x, y-1)$$

This implies that

$$\nabla^2 f = [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)] - 4f(x, y)$$

The 3×3 Laplacian operator is given by.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- The Laplacian operator subtracts the brightness values of each of the neighbouring pixels from the central pixel.
- The Laplacian operator is rotationally invariant.
- The Laplacian operator does not depend on directions as long as they are orthogonal.

Drawbacks of the Laplacian Operator:

1. Useful directional information is not available by the use of Laplacian operator.
2. The Laplacian, being an approximation to the second derivative, doubly enhances any noise in the image.

THRESHOLDING

Thresholding is the simplest method of image segmentation. It replaces each pixel in an image with a black pixel if the image intensity is less than a fixed value called the threshold or a white pixel if the pixel intensity is greater than that threshold.

The basics of intensity thresholding

Suppose that the intensity histogram corresponds to an image, $f(x, y)$, composed of light objects on a dark background, in such a way that object

and background pixels have intensity values grouped in to two dominant modes.

The segmented image $g(x, y)$ is given by.

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T : (x, y) \text{ is an object point.} \\ 0 & \text{if } f(x, y) \leq T : \text{the Point is a background point} \end{cases}$$

When T is a constant applicable over an entire image, the process given in the above equation is referred to as global thresholding.

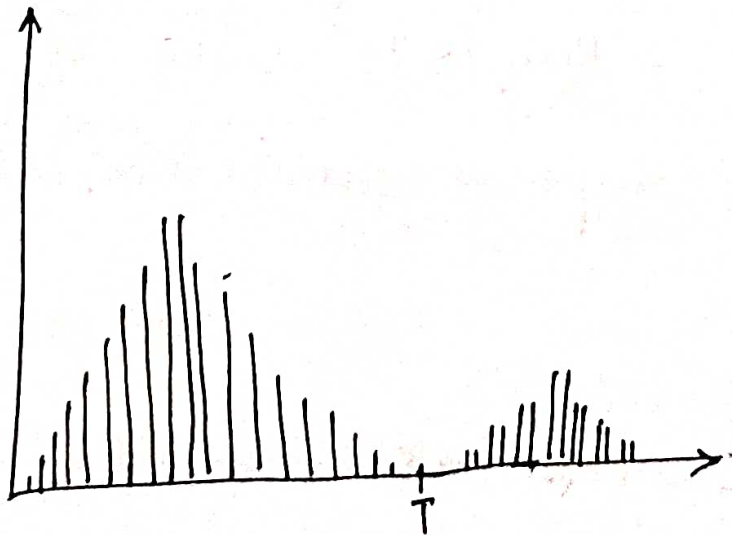


Fig: Intensity threshold (partitioning of single value)

Local or regional thresholding: is used to denote variable thresholding in which the value of T at any point (x, y) in an image depends on properties of a neighborhood of (x, y) .

Dynamic or Adaptive thresholding: if T depends on the spatial coordinates (x, y) themselves then variable thresholding is often referred to as dynamic or adaptive.

Multiple thresholding:

$$g(x, y) = \begin{cases} a & \text{if } f(x, y) > T_2 : \text{Other Object class.} \\ b & \text{if } T_1 < f(x, y) \leq T_2 : \text{One object} \\ & \text{class.} \\ c & \text{if } f(x, y) \leq T_1 : \text{Background.} \end{cases}$$

Where a, b, c are any three distinct intensity values.

BASIC GLOBAL THRESHOLDING

The following algorithm (iterative) can be used for basic global thresholding:



1. Select an initial estimate for the global threshold, T .

2. Segment the image using T in eqn.

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T \end{cases}$$

This will produce two groups of pixels:

G_1 consisting of all pixels with intensity values $> T$.

G_2 consisting of pixels with values $\leq T$.

3. Compute the average (mean) intensity values m_1 and m_2 for the pixels in G_1 and G_2 , respectively.

4. Compute a new threshold value:

$$T = \frac{1}{2} (m_1 + m_2)$$

5. Repeat Steps 2 through 4 until the difference between values of T in successive iterations is smaller than a predefined parameter ΔT .

ΔT is used to control the number of iterations in situation where speed is an important issue.

OPTIMUM GLOBAL THRESHOLDING

USING OTSU'S METHOD

The Otsu's approach is optimum in the sense that it maximizes the between-class variance, a well-known measure used in statistical discriminant analysis.

* Here the well-thresholded classes should be distinct with respect to the intensity values of their pixel.

* Also, this method has the important property that this is based on computations performed on the histogram of an image, an easily obtainable 1-D array.

Let $\{0, 1, 2, \dots, L-1\}$ denote the L distinct intensity levels in a digital image of size $M \times N$.

Let n_i denote the number of pixels with intensity i .

The normalized histogram has component

$$p_i = n_i / MN \text{ that follows.}$$

$$\sum_{i=0}^{L-1} P_i = 1, \quad P_i \geq 0$$

Suppose, we select a threshold $T(k) = k$, $0 \leq k \leq L-1$, and use ^{it to} the threshold the i/p image into two classes C_1 and C_2 .

Where C_1 consists of all the pixels in the image with intensity values in the range $[0, k]$

C_2 consists of the pixels with values in the range $[k+1, L-1]$.

Otsu's algorithm may be summarized as follows:

1: Compute the normalized histogram of the input image. Denote the components of the histogram by P_i , $i = 0, 1, 2, \dots, L-1$.

2: Compute the cumulative sums, $P_1(k)$, for $k = 0, 1, 2, \dots, L-1$, using the equation.

$$P_1(k) = \sum_{i=0}^k P_i$$

3: Compute the cumulative means $m(k)$, (avg intensity) for $k=0, 1, 2, \dots, L-1$ using Equation

$$m(k) = \sum_{i=0}^k i P_i$$

4: Compute the global intensity mean, m_G , using equation (ie, the global mean).

$$m_G = \sum_{i=0}^{L-1} i P_i$$

5: Compute between-class variance $\sigma_B^2(k)$, for $k=0, 1, 2, \dots, L-1$ using equation.

$$\sigma_B^2(k) = \frac{[m_G P_1(k) - m(k)]^2}{P_1(k) [1 - P_1(k)]}$$

6: Obtain the Otsu threshold k^* , as the value of k for which $\sigma_B^2(k)$ is maximum. If the maximum is not unique, obtain k^* by averaging the values of k corresponding

to the various maxima detected.

7: Obtain the separability measure, η^* , by evaluating equation given below at $k=k^*$.

Once k^* has been obtained, the input image $f(x,y)$ is segmented as before:

$$g(x,y) = \begin{cases} 1 & \text{if } f(x,y) > k^* \\ 0 & \text{if } f(x,y) \leq k^* \end{cases}$$

for $x=0, 1, 2, \dots, M-1$ &

$y=0, 1, 2, \dots, N-1$.

MULTIPLE THRESHOLDS

Threshold can be extended to an arbitrary number of thresholds, because the separability measure on which it is based also extends to an arbitrary number of classes. In the case of K classes, $C_1, C_2, \dots, C_K$ the between-class variance generalizes to the expression:

$$\sigma_B^2 = \sum_{k=1}^K P_k (m_k - m_G)^2.$$

Where

$$P_k = \sum_{i \in C_k} P_i$$

$$m_k = \frac{1}{P_k} \sum_{i \in C_k} i P_i$$

For three classes consisting of three intensity intervals the between-class variance is given by:

$$\sigma_B^2 = P_1 (m_1 - m_G)^2 + P_2 (m_2 - m_G)^2 + P_3 (m_3 - m_G)^2$$

Where

$$P_1 = \sum_{i=0}^{k_1} P_i$$

$$P_2 = \sum_{i=k_1+1}^{k_2} P_i$$

$$P_3 = \sum_{i=k_2+1}^{L-1} P_i$$

and

$$m_1 = \frac{1}{P_1} \sum_{i=0}^{k_1} i P_i$$

$$m_2 = \frac{1}{P_2} \sum_{i=k_1+1}^{k_2} i P_i$$

$$m_3 = \frac{1}{P_3} \sum_{i=k_2+1}^{L-1} i P_i$$

The following relationships hold:

$$P_1 m_1 + P_2 m_2 + P_3 m_3 = m_G$$

and $P_1 + P_2 + P_3 = 1.$

The two optimum threshold values k_1^* and k_2^* are the values that maximize $\sigma_B^2(k_1, k_2)$

Optimum threshold,

$$\sigma_B^2(k_1^*, k_2^*) = \max_{0 < k_1 < k_2 < L-1} \sigma_B^2(k_1, k_2)$$

The threshold image is given by:

$$g(x, y) = \begin{cases} a & \text{if } f(x, y) \leq k_1^* \\ b & \text{if } k_1^* < f(x, y) \leq k_2^* \\ c & \text{if } f(x, y) > k_2^* \end{cases}$$

Where a, b and c are any three intensity values.

Multiple thresholds:

$$\eta(k_1^*, k_2^*) = \frac{\sigma_B^2(k_1^*, k_2^*)}{\sigma_G^2}$$

Where σ_G^2 is the total image variance.

VARIABLE THRESHOLDING

Various techniques for choosing variable thresholds are discussed in the following subsections.

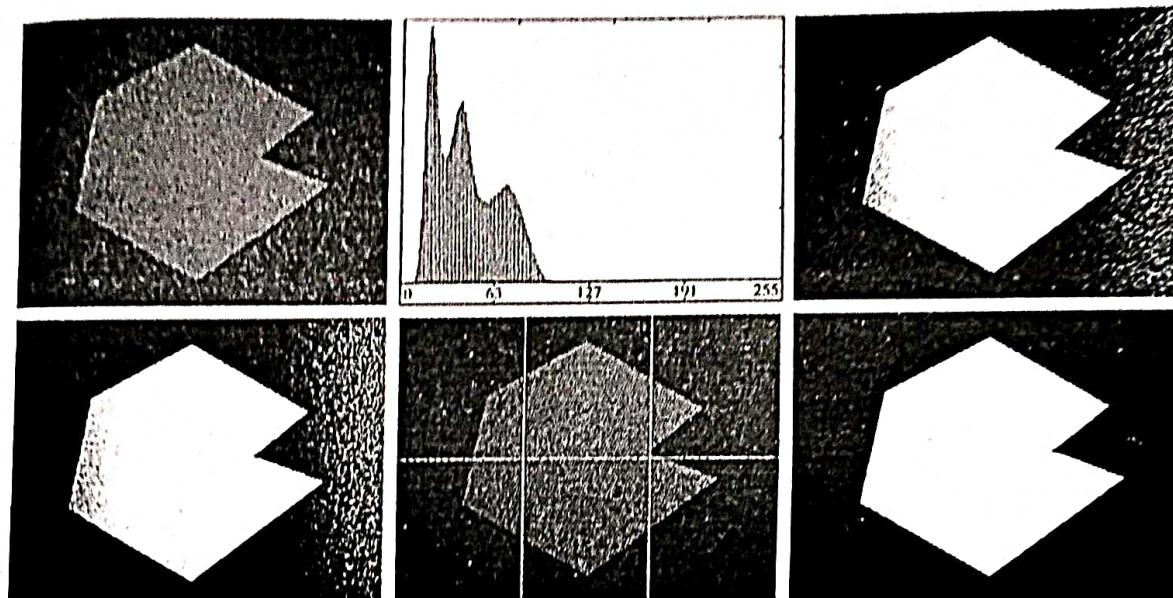
1. Image Partitioning

- ★ One of the simplest approaches to variable thresholding is to subdivide an image into nonoverlapping rectangles.

- ★ Image partitioning is used to compensate for non-uniformities in illumination and/or reflectance.

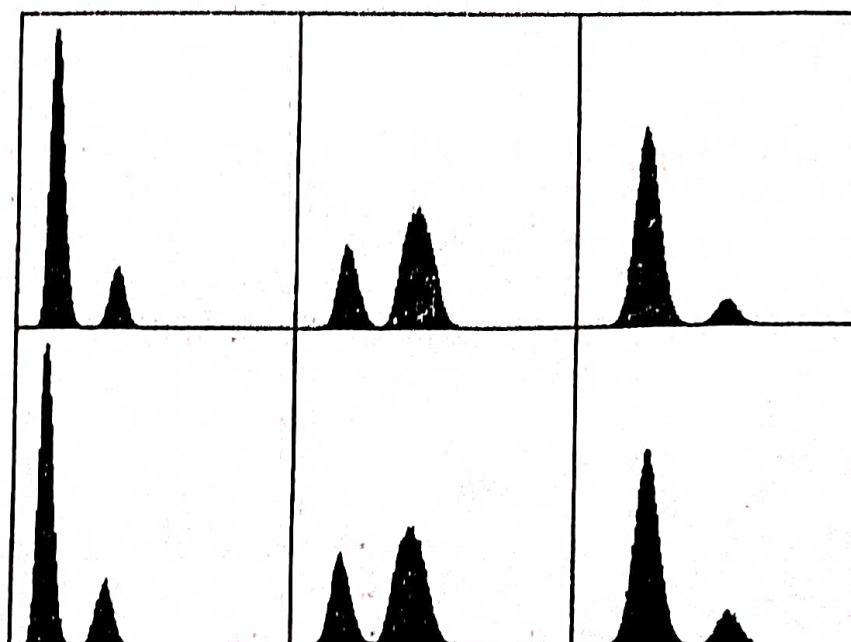
- ★ The rectangles are chosen small enough so that the illumination of each is approximately uniform.

The image partitioning approach is illustrated in the following figure. Image subdivision generally works well when the objects of interest and the background occupy regions of reasonably comparable size.



a b c
d e f

(a) Noisy, shaded image and (b) its histogram. (c) Segmentation of (a) using the iterative global algorithm from Section 10.3.2. (d) Result obtained using Otsu's method. (e) Image subdivided into six subimages. (f) Result of applying Otsu's method to each subimage individually.



Histograms of the
six subimages in
Fig. (c).

2. Variable thresholding based on local image properties.

Let σ_{xy} and m_{xy} denote the standard deviation and mean value of the set of pixels contained in a neighborhood, S_{xy} , centered at coordinates (x, y) in an image.

Following are common forms of variable, local thresholds:

$$T_{xy} = a\sigma_{xy} + b m_{xy}$$

Where a and b are nonnegative constants, and

$$T_{xy} = a\sigma_{xy} + b m_G$$

Where m_G is the global image mean.

The segmented image is computed as

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T_{xy} \\ 0 & \text{if } f(x, y) \leq T_{xy} \end{cases}$$

Where $f(x, y)$ is the input image.

Different threshold is computed at each location (x, y) using the pixels in the neighborhood S_{xy} .

$$g(x,y) = \begin{cases} 1 & \text{if } Q(\text{local parameters}) \text{ is true} \\ 0 & \text{if } Q \quad \quad \quad \text{is false} \end{cases}$$

Where Q is a predicate based on parameters computed using the pixels in neighborhood S_{xy} .

$$Q(\sigma_{xy}, m_{xy}) = \begin{cases} \text{true} & \text{if } (x,y) > a\sigma_{xy} \text{ AND } f(x,y) > b m_{xy} \\ \text{false} & \text{otherwise} \end{cases}$$

3. Using moving averages

Let z_{k+1} denote the intensity of the point encountered in the scanning sequence at step $k+1$.

The moving average (mean intensity) at this new point is given by

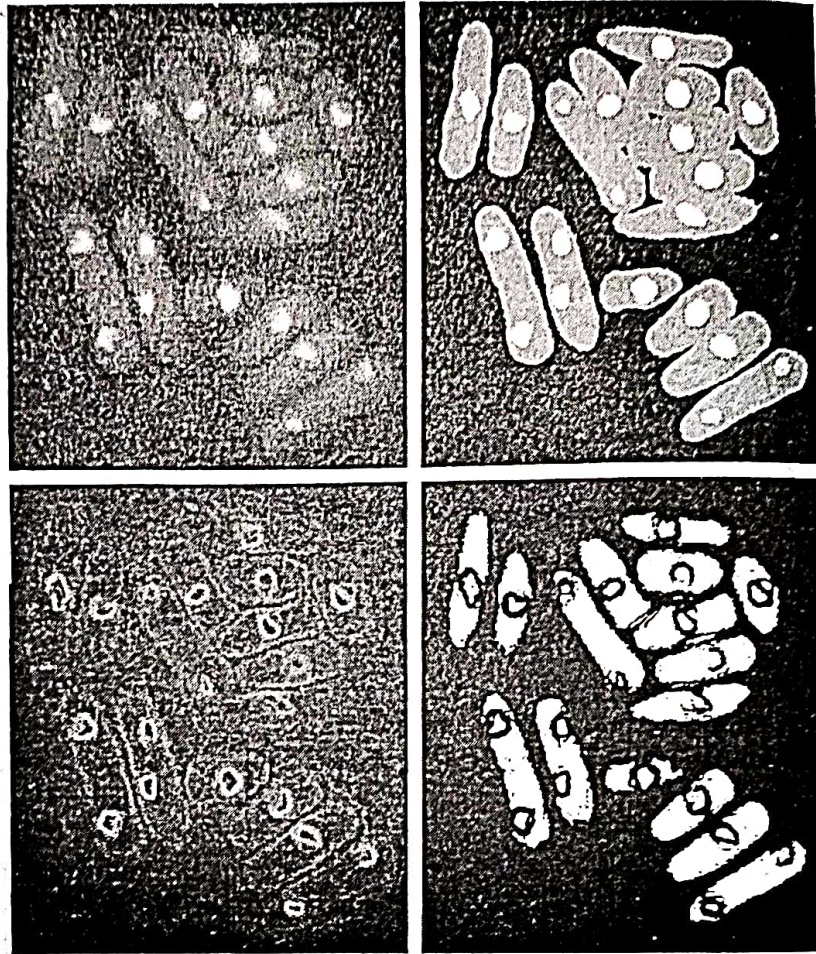
$$m(k+1) = \frac{1}{n} \sum_{i=k+2-n}^{k+1} z_i$$

$$= m(k) + \frac{1}{n} (z_{k+1} - z_{k-n})$$

Where n denotes the number of points used in computing the average and $m(1) = z_1/n$.

a b
c d

(a) Image from Fig. 10.43.
(b) Image segmented using the dual thresholding approach discussed in Section 10.3.6.
(c) Image of local standard deviations.
(d) Result obtained using local thresholding.



MULTIVARIABLE THRESHOLDING

Multivariable thresholding may be viewed as a distance computation. Let a denote the average reddish color in which we are interested. One way to segment a colour image based on this parameter is to compute a distance measure,

$D(z, a)$, between an arbitrary color point, z , and the average colour, a , then we segment the input image as follows:

$$g = \begin{cases} 1 & \text{if } D(z, a) < T \\ 0 & \text{Otherwise.} \end{cases}$$

n -dimensional Euclidean distance is defined as

$$\begin{aligned} D(z, a) &= \|z - a\| \\ &= [(z - a)^T (z - a)]^{1/2} \end{aligned}$$

The equation $D(z, a) = T$ describes a sphere (called a sphere) in n -dimensional Euclidean space.

A more powerful distance measure is the so called Mahalanobis distance, defined as

$$D(z, a) = [(z - a)^T C^{-1} (z - a)]^{1/2}$$

Where C is the covariance matrix of the z s.

$D(z, a) = T$ describes an n -dimensional hyperellipse.

When $C = I$, the identity matrix.

REGION-BASED APPROACH TO SEGMENTATION

This section discuss segmentation techniques that are based on finding the regions directly.

1. Region Growing

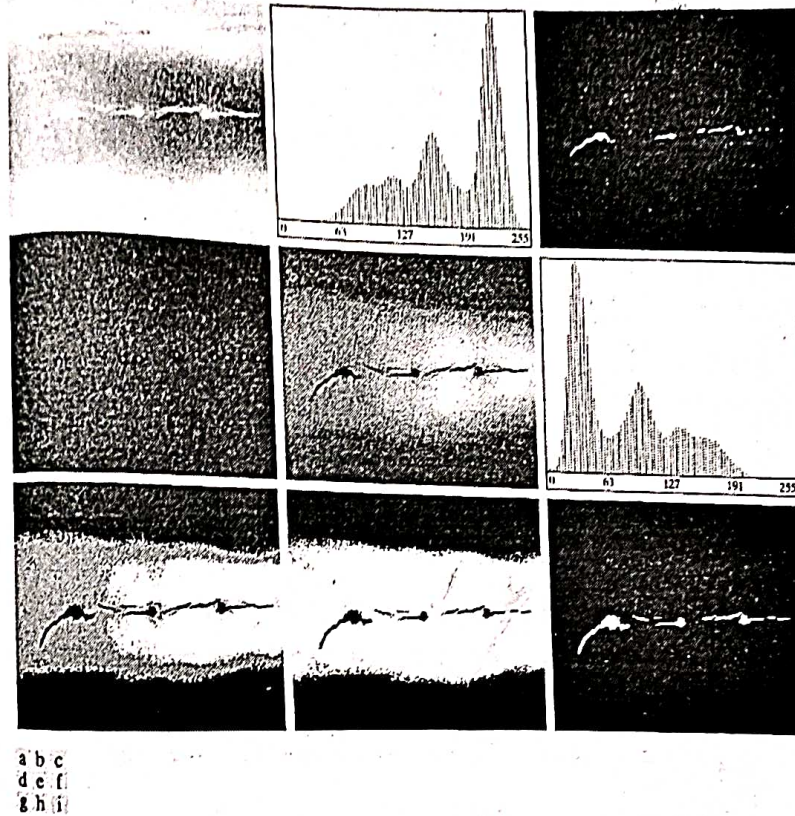
As its name implies, region growing is a procedure that groups pixels or subregions into larger regions based on predefined criteria for growth. The basic approach is to start with a set of "seed" points and from these grow regions by appending to each seed those neighboring pixels that have predefined properties similar to the seed.

Let: $f(x, y)$ denote an input image array;
 $s(x, y)$ denote a seed array containing 1s at the locations of seed points and 0s elsewhere;
and Q denote a predicate to be applied at each location (x, y) . Arrays f and s are assumed to be of the same size. A basic region-growing algorithm based on 8-connectivity may be stated as follows:

1. Find all connected components in $S(x,y)$ and erode each connected component to one pixel; label all such pixels found as 1. All other pixels in S are labeled 0.
2. Form an image f_Q such that, at a pair coordinates (x,y) , let $f_Q(x,y)=1$ if the input image satisfies the given predicate, Q , at those coordinates; otherwise, let $f_Q(x,y)=0$.
3. Let g be an image formed by appending to each seed point in S all the 1-valued points in f_Q that are 8-connected to that seed point.
4. Label each connected component in g with a different region label (eg:- 1, 2, 3, ...). This is the segmented image obtained by region growing.

The region growing mechanics of this is illustrated by an example shown in the following figure.

$$Q = \begin{cases} \text{TRUE} & \text{if the absolute difference of the} \\ & \text{intensities between the seed and} \\ & \text{the pixel at } (x,y) \text{ is } \leq T. \\ \text{FALSE} & \text{Otherwise.} \end{cases}$$



(a) X-ray image of a defective weld. (b) Histogram. (c) Initial seed image. (d) Final seed image (the points were enlarged for clarity). (e) Absolute value of the difference between (a) and (c). (f) Histogram of (e). (g) Difference image thresholded using dual thresholds. (h) Difference image thresholded with the smallest of the dual thresholds. (i) Segmentation result obtained by region growing. (Original image courtesy of X-TEK Systems, Ltd.)

2. Region Splitting and Merging

Terminologies used in this:

Quadtree: trees in which each node has exactly four descendants. Quadtree sometimes

are called quadregions or quadimages.

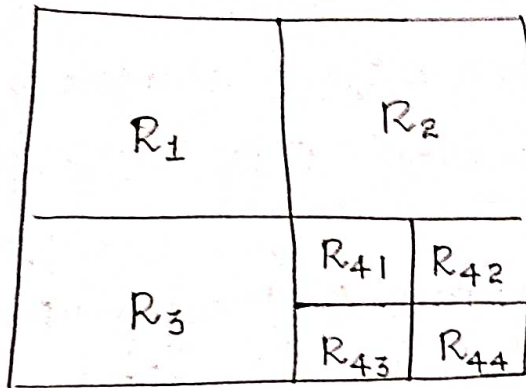


Fig:- Partitioned Image.

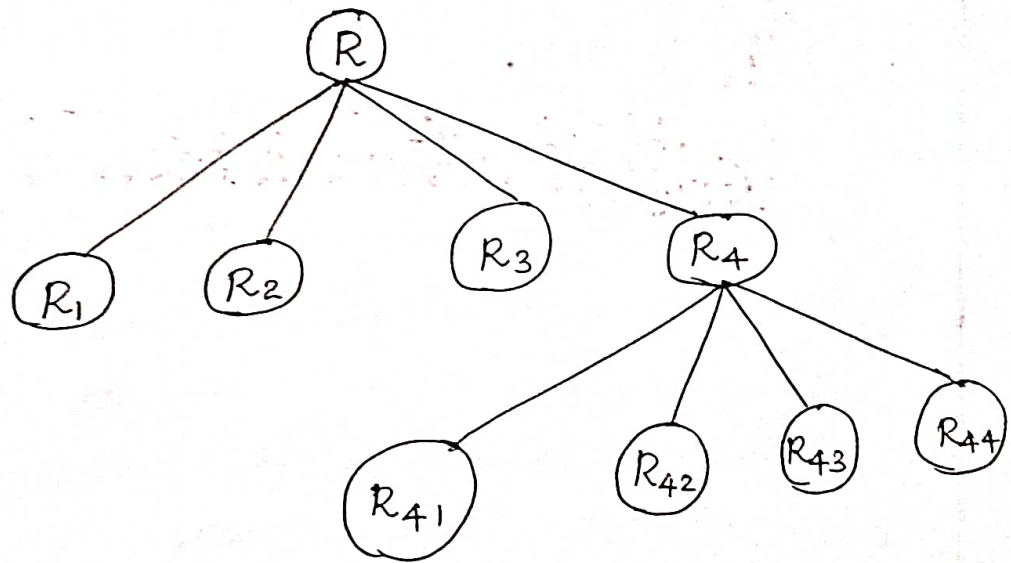


Fig:- Corresponding quadtree. R represents the entire image region.

The region splitting and merging can be summarized as follows:

1. Split into four disjoint quadrants any region R_i for which $Q(R_i) = \text{FALSE}$.
2. When no further splitting is possible, merge any adjacent regions R_j and R_k for which $Q(R_j \cup R_k) = \text{TRUE}$.
3. Stop when no further merging is possible.